



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Testspezifikation

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 6. Juli 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Mohamed Sabri Jlizi	Aus der Abnahmetestspezifikation
2	Lucas Neidahl, Mohammad Khatib	Aus der Abnahmetestspezifikation
2.1	Lucas Neidahl	Aus der Abnahmetestspezifikation
2.2	Lucas Neidahl	Aus der Abnahmetestspezifikation
2.3	Lucas Neidahl	Aus der Abnahmetestspezifikation
2.4	Mohammad Khatib, Luan Hyseni	Aus der Abnahmetestspezifikation
2.5	Mohammad Khatib	Aus der Abnahmetestspezifikation
3	Mohammed Alhalabi	Aus der Abnahmetestspezifikation
3.1	Mohammed Alhalabi	Aus der Abnahmetestspezifikation
3.2	Mohammed Alhalabi	Aus der Abnahmetestspezifikation
3.3	Mohammed Alhalabi	Aus der Abnahmetestspezifikation
4	Mohamed Sabri Jlizi	Fertig
4.1	Mohamed Sabri Jlizi	Fertig
4.2	Mohamed Sabri Jlizi	Fertig
4.3	Mohamed Sabri Jlizi, Mohammad Khatib	Fertig
5	Marvin Voigt	Fertig (nochmal prüfen)
5.1	Marvin Voigt	Fertig (nochmal prüfen)
5.2	Marvin Voigt	Fertig (nochmal prüfen)
5.3	Marvin Voigt	Fertig (nochmal prüfen)
6	Mohammed Alhalabi	Fertig (nochmal prüfen)

Inhaltsverzeichnis

1	Einleitung	6
2	Testplan	7
2.1	Zu testende Komponenten	7
2.2	Zu testende Funktionen/Merkmale	8
2.3	Nicht zu testende Funktionen	11
2.4	Vorgehen	11
2.4.1	Komponententests	11
2.4.2	Integrationstest	11
2.4.3	Systemtest	12
2.4.4	Abnahmetest	12
2.5	Testumgebung	12
3	Abnahmetest	14
3.1	Zu testende Anforderungen	15
3.2	Testverfahren	16
3.2.1	Testskripte	16
3.3	Testfälle	17
3.3.1	Testfall $\langle T100 \rangle$ - Szene erstellen und speichern	18
3.3.2	Testfall $\langle T200 \rangle$ - Simulation einer Drohnenshow	19
4	Integrationstest	22
4.1	Zu testende Komponenten	22
4.2	Testverfahren	23
4.2.1	Testskripte	23
4.3	Testfälle	23
4.3.1	Testfall $\langle T500 \rangle$ - Komponente $\langle C10+C20 \rangle$	24
4.3.2	Testfall $\langle T510 \rangle$ - Komponente $\langle C20+C60 \rangle$	25
4.3.3	Testfall $\langle T520 \rangle$ - Komponente $\langle C10+C60 \rangle$	26
4.3.4	Testfall $\langle T530 \rangle$ - Komponente $\langle C10+C30 \rangle$	27
4.3.5	Testfall $\langle T540 \rangle$ - Komponente $\langle C50+C60 \rangle$	28
4.3.6	Testfall $\langle T550 \rangle$ - Komponente $\langle C20+C70 \rangle$	29
4.3.7	Testfall $\langle T560 \rangle$ - Komponente $\langle C20+C30+C60+C90 \rangle$	30

5	Unit-Tests	31
5.1	Zu testende Komponenten	31
5.2	Testverfahren	32
5.2.1	Testskripte	32
5.3	Testfälle	32
5.3.1	Testfall $\langle T600 \rangle$ - ShowKonfigurator: Szene anlegen	33
5.3.2	Testfall $\langle T601 \rangle$ - ShowKonfigurator: Szene bearbeiten	34
5.3.3	Testfall $\langle T610 \rangle$ - Projektmanagement: Projekt speichern und laden	35
5.3.4	Testfall $\langle T611 \rangle$ - Projektmanagement: Projekt löschen	36
5.3.5	Testfall $\langle T620 \rangle$ - Simulation: Szenenübergabe	37
5.3.6	Testfall $\langle T630 \rangle$ - Validierung: Eingabefehler erkennen	38
5.4	Zu testende Komponenten	39
5.5	Testverfahren	39
5.5.1	Testskripte	39
5.6	Testfälle	39
5.6.1	Testfall $\langle T300 \rangle$ - Klasse x	40
6	Glossar	41

Abbildungsverzeichnis

1 Einleitung

Im Allgemeinen ist das Testen von Software ein wesentlicher Schritt im Entwicklungsprozess, da es zur Sicherstellung der Qualität beiträgt. Mit systematischem Testen können Probleme schon frühzeitig identifiziert und gelöst werden. Auf diese Weise können Verzögerungen im Projektverlauf vermieden und die im Pflichtenheft festgelegten Anforderungen zuverlässig umgesetzt werden. Vor allem bei Projekten mit komplizierten Nutzerinteraktionen und grafischen Simulationen – wie bei SkyOrchestrator – ist es essenziell, von Anfang an und fortlaufend Tests durchzuführen, um eine stabile und benutzerfreundliche Anwendung sicherzustellen. Diese Abnahmetestspezifikation betrifft die Software SkyOrchestrator, die im Sommersemester 2025 im Rahmen eines Softwareentwicklungspraktikums an der Technischen Universität Braunschweig entwickelt werde. Das Ziel ist, eine benutzerfreundliche Plattform zur Planung, Simulation und Visualisierung von Drohnenshows zu entwickeln. Ziel ist es, dass die Software sich an Kreative ohne Programmierkenntnisse sowie an Profi-Eventplaner richtet, damit diese auf einfache Weise komplexe Drohnenchoreografien entwerfen und simulieren können. Die Tests richten sich nach dem IEEE 829-Standard für Software-Testdokumente. Dieser Standard definiert eine klare Struktur für Testberichte und ermöglicht so eine nachvollziehbare Qualitätssicherung. SkyOrchestrator muss bestimmte Kriterien erfüllen, insbesondere in Bezug auf Funktionen, Benutzerfreundlichkeit und Erweiterbarkeit. Es soll eine flüssige 3D-Simulation basierend auf den Eingaben der Nutzer bieten, eine intuitive Benutzeroberfläche haben und visuelle Formationen sowie Lichteffekte präzise darstellen. Die Technologie nutzt Python 3.13 und die Simulationsbibliothek PyBullet, wobei Windows 11 die Zielplattform ist. Bei den Abnahmetests soll kontrolliert werden, ob die Software alle definierten Anforderungen erfüllt. Typische Nutzungsszenarien, Funktionsaufrufe und das Systemverhalten unter unterschiedlichen Bedingungen werden getestet. Die Software wird erst dann als fehlerfrei und bereit für die Abnahme durch den Auftraggeber angesehen, wenn alle Tests bestanden wurden.

2 Testplan

Im folgenden Kapitel wird der Testplan für den Skyorchester beschrieben. Dabei wird sowohl auf die Komponenten und Funktionen eingegangen, die getestet werden sollen, als auch auf solche, die nicht getestet werden. Außerdem wird die allgemeine Vorgehensweise zum Testen der Funktionen beschrieben und erklärt.

2.1 Zu testende Komponenten

Die folgenden Hauptkomponenten unserer Software werden getestet:

- **GUI** Benutzeroberfläche zur Nutzerinteraktion mit der Software. Der Nutzer kann hier die Drohnenshow konfigurieren, simulieren und Sie verwalten. Hierbei muss getestet werden:
 - Richtige Darstellung und intuitive Bedienung ohne Programmierkenntnisse
 - Fehlerfreie Eingabeverarbeitung und visuelle Rückmeldungen.
 - Funktionalität der Timeline-Komponente.
- **Show-Konfigurationsmodul** Es verarbeitet die Nutzereingaben zur Erstellung und Verwaltung der Show-Logik. Hierbei muss getestet werden:
 - Übernahme, Validierung und Speicherung von Show-Parametern.
 - Konsistente Verwaltung von Szenen und Formationswechseln.
 - Vollständige Datenaufbereitung für die Simulation
- **Simulations-Engine** Berechnet Drohnenbewegungen, Lichteffekte und stellt die 3D-Umgebung dar. Hierbei muss getestet werden:
 - Korrekte Umsetzung der Choreographie-Parameter und Lichteffekte
 - Performance und Skalierbarkeit
- **Projektmanagement-Modul** Zuständig für das Speichern und Laden von Drohnenshow-Projekten. Hierbei muss getestet werden:

- Fehlerfreies Speichern und Laden von Projekten.
- Datenintegrität nach Speichern und Laden.
- Korrekter Umgang mit fehlerhaften Projektdateien.

2.2 Zu testende Funktionen/Merkmale

Hier geht es um alle Funktionen und Eigenschaften, die wir uns beim Testen genau ansehen wollen. Wir listen hier alles auf, was getestet werden soll und beziehen uns dabei auf die Funktions-IDs aus dem Pflichtenheft. Außerdem werden auch hier die Messkriterien getestet. Die Software SkyOrchestrator wird als quelloffenes Python-Projekt über ein zentrales GitLab-Repository bereitgestellt. Die Anwendung liegt am Ende als .exe Datei vor. Die Anwendung wird sowohl in der .exe Form als auch in Form von Quellcode, Skripten und Konfigurationsdateien vom Git-Repository getestet.

Der Zugriff auf das Git Repository benötigt eine Internetverbindung und Speicherplatz für das Klonen des Repositories. Die Installation der Python-Umgebung und der Abhängigkeiten benötigt ebenfalls etwa 500MB Speicherplatz.

Vor der Durchführung der Tests muss die Software wie folgt vorbereitet werden:

- Klonen des aktuellen Entwicklungsstandes aus dem GitLab-Repository.
- Einrichtung einer Python 3.13 Umgebung (z.B. mittels venv).
- Installation aller erforderlichen Abhängigkeiten, die in einer requirements.txt-Datei im Repository spezifiziert sind (insbesondere PyBullet und die für Tkinter benötigten Systembibliotheken, falls nicht bereits vorhanden). Dies erfolgt typischerweise mit dem Befehl `pip install -r requirements.txt` innerhalb der aktivierten virtuellen Umgebung.
- Die Software wird direkt aus dem Quellcode durch Ausführen des Hauptskripts gestartet. Eine separate Kompilierung ist nicht notwendig."

Folgende Hauptfunktionen der Anwendung werden getestet:

(F 10) Konfiguration der Show

- Der Benutzer plant die Inhalte einer Drohnenshow durch Erstellung, Bearbeitung und Verwaltung einzelner Szenen. Die Drohnenshow besteht aus den vorher konfigurierten Szenen mit definierter Bewegung, Beleuchtung und Zeitsteuerung, die zur Simulation übergeben werden können. Alle grundlegenden Planungsfunktionen müssen so gestaltet sein, dass die Bedienung ohne Programmierkenntnisse möglich ist (Testaspekt für RM4).

- Zugehörige Muss-Kriterien für die Gesamtfunktion $\langle F\ 10 \rangle$: $\langle RM1 \rangle$, $\langle RM2 \rangle$, $\langle RM3 \rangle$, $\langle RM4 \rangle$, $\langle RM5 \rangle$, $\langle RM6 \rangle$, $\langle RM7 \rangle$.
- Die Konfiguration der Show (F10) unterteilt sich in folgende spezifische Funktionalitäten, die detailliert getestet werden:
 - **$\langle F11 \rangle$ Szene anlegen und Metadaten erfassen**
 - **$\langle F12 \rangle$ Formation und Bewegung definieren**
 - **$\langle F13 \rangle$ Licht- und Zeiteffekte konfigurieren**
 - **$\langle F14 \rangle$ Szene speichern und zur Simulation übergeben**

⟨F 20⟩ Choreographie simulieren

- Eine zuvor konfigurierte Drohnenshow wird in der Simulation abgespielt. Die geplante Choreographie wird in einer 3D-Umgebung möglichst flüssig und in Echtzeit simuliert.
- Zugehörige Muss-Kriterien: ⟨RM1⟩, ⟨RM4⟩, ⟨RM5⟩

⟨F 30⟩ Projekt verwalten

- Ein Projekt wird gespeichert, geladen oder exportiert. Der Benutzer kann eine bestehende Konfiguration speichern, wiederherstellen oder in ein Austauschformat exportieren.
- Zugehörige Muss-Kriterien: ⟨RM4⟩, ⟨RM5⟩

⟨F 40⟩ Szenenprüfung auf Eingabefehler

- Der Benutzer überprüft eine Szene vor dem Speichern oder der Simulation. Das System erkennt fehlende Pflichtangaben oder ungültige Werte und zeigt sie gesammelt an. Fehlerhafte oder unvollständige Eingaben sollen dem Benutzer klar gemeldet werden, damit diese korrigiert werden können.

Zusätzlich zu den funktionalen Anforderungen werden auch die folgenden Qualitätsanforderungen (gemäß Pflichtenheft Kapitel 6.5) im Rahmen der Tests überprüft, um die allgemeine Güte und Gebrauchstauglichkeit der Software sicherzustellen:

- **⟨Q10⟩ Learnability:** Beherrschbarkeit der Grundfunktionen binnen 15 Minuten ohne Anleitung.
- **⟨Q20⟩ Effizienz:** Erreichbarkeit häufig genutzter Funktionen mit maximal zwei Klicks.
- **⟨Q30⟩ Fehlertoleranz (UI):** Unterstützung bei Eingabefehlern durch Validierung, Vorschläge, Korrekturhinweise.
- **⟨Q40⟩ Ressourcenverbrauch:** Speicherverbrauch unter 1 GB RAM bei Simulation mit 50 Drohnen.
- **⟨Q50⟩ Dokumentation:** Verfügbarkeit kontextsensitiver Hilfetexte und Tooltips für Hauptfunktionen.
- **⟨Q60⟩ Reaktionszeit UI:** Reaktionen auf Benutzeraktionen (außerhalb Simulation) unter 1 Sekunde.
- **⟨Q70⟩ Plattformkonformität:** Lauffähigkeit auf Python 3.11+ unter Windows 10/11 ohne Codeänderungen.
- **⟨Q80⟩ Fehlertoleranz (Eingaben):** Erkennung ungültiger Benutzereingaben und verständliche Fehlermeldung innerhalb 1 Sekunde.

2.3 Nicht zu testende Funktionen

Alle in dem Kapitel Produktfunktionen beschriebenen Funktionalitäten aus dem Pflichtenheft ($\langle F 10 \rangle$, $\langle F 11 \rangle$, $\langle F 12 \rangle$, $\langle F 13 \rangle$, $\langle F 20 \rangle$, $\langle F 30 \rangle$, $\langle F 40 \rangle$) werden von uns getestet. Außerdem werden die Qualitätsanforderungen Q10, Q20, Q30, Q40, Q50, Q60, Q70, Q80 ebenfalls überprüft.

2.4 Vorgehen

In diesem Abschnitt wird das grundsätzliche Vorgehen zur Durchführung der Softwaretests erläutert. Ziel ist es, sicherzustellen, dass alle wesentlichen Funktionen von SkyOrchestrator zuverlässig und korrekt arbeiten. Die Testmethodik orientiert sich am V-Modell und unterscheidet vier aufeinander aufbauende Teststufen:

2.4.1 Komponententests

Die in Kapitel 2.1 beschriebenen Hauptkomponenten von SkyOrchestrator werden in dieser Teststufe einzeln und unabhängig voneinander getestet. Dabei werden gezielt einzelne Module isoliert geprüft. Die Tests erfolgen automatisiert mit dem Python-Framework pytest. Für jede Funktionseinheit werden definierte Testfälle implementiert, um sowohl korrekte als auch fehlerhafte Eingaben zu überprüfen. Die erwarteten Ausgaben werden mit den tatsächlichen Ergebnissen verglichen und systematisch protokolliert.

2.4.2 Integrationstest

Nach Abschluss der Unit-Tests werden die einzelnen Komponenten von SkyOrchestrator schrittweise zu größeren Funktionseinheiten zusammengeführt. Dabei wird das Bottom-Up-Prinzip angewendet: Zunächst werden kleinere Module miteinander verbunden, anschließend folgen Tests auf höheren Ebenen, etwa zwischen Konfigurationslogik, Simulationsmodul und Projektverwaltung. Die Tests werden auf Basis definierter Anwendungsfälle durchgeführt und die Ergebnisse schriftlich protokolliert. Durch die frühzeitige Überprüfung von Schnittstellen und Modulverbindungen können Probleme in späteren Testphasen vermieden werden.

2.4.3 Systemtest

Nach erfolgreichem Abschluss der Integrationstests wird das Gesamtsystem SkyOrchestrator als vollständige Anwendung getestet. Ziel ist es, sowohl funktionale als auch nicht-funktionale Anforderungen unter realitätsnahen Bedingungen zu prüfen. Die Tests finden in einer Testumgebung statt, die der geplanten Zielumgebung beim Nutzer möglichst genau entspricht.

Im Mittelpunkt stehen hier Black-Box-Tests: Das Systemverhalten wird ausschließlich aus Sicht der Anwender:innen geprüft, ohne auf interne Implementierungsdetails einzugehen. Alle Hauptfunktionen – von der Show-Konfiguration über die Simulation bis zur Projektverwaltung – werden getestet. Zusätzlich wird auf Aspekte wie Benutzerfreundlichkeit, Reaktionszeit und Stabilität geachtet.

Die Testfälle basieren auf den Anforderungen aus dem Pflichtenheft. Die Ergebnisse werden dokumentiert und bei Fehlern durch Screenshots, Logdateien oder direkte Beobachtungen ergänzt. Der Systemtest stellt damit sicher, dass die Software im geplanten Nutzungskontext zuverlässig funktioniert und für den Kunden einsatzbereit ist.

2.4.4 Abnahmetest

Nach Abschluss aller vorangegangenen Teststufen wird der Abnahmetest durchgeführt. Ziel ist es, die Produktreife aus Sicht des Auftraggebers zu bestätigen und die im Pflichtenheft definierten Anforderungen vollständig zu überprüfen. Die Tests werden in einer produktionsnahen Umgebung durchgeführt und orientieren sich strikt an den funktionalen Muss-Kriterien. Der Abnahmetest dient nicht mehr primär der Fehlersuche, sondern der formellen Bestätigung, dass die Software vollständig und vertragsgemäß geliefert wurde. Dafür wird SkyOrchestrator mithilfe typischer Anwendungsszenarien getestet, beispielsweise durch das Anlegen, Speichern und Simulieren komplexer Shows.

2.5 Testumgebung

Als Testumgebung kommt ein lokal eingerichtetes Testsystem unter Windows 11 zum Einsatz, das der Zielpattform der Software entspricht. Für die automatisierten Komponententests wird die Python-Testbibliothek pytest verwendet. Die Tests laufen auf einem leistungsfähigen Desktop-PC mit folgender Ausstattung:

- Intel Core i7-7700K CPU
- 32 GB RAM

- NVIDIA RTX 2080 GPU
- SSD-Speicher

Zur Validierung exportierter Daten werden JSON-Validatoren und Texteditoren wie VS Code und Notepad++ verwendet. Für die Fehleranalyse werden zusätzlich Logdateien sowie Bildschirmaufzeichnungen herangezogen. Diese Umgebung ermöglicht es, sowohl automatisierte Unit-Tests als auch realitätsnahe manuelle Blackbox-Tests unter produktionsnahen Bedingungen durchzuführen.

3 Abnahmetest

In diesem Kapitel werden die funktionalen Anforderungen der Software SkyOrchestrator anhand festgelegter Testfälle überprüft. Die im Pflichtenheft festgelegten Muss-Kriterien und Kernfunktionen bilden die Grundlage für die Tests, deren Ziel es ist, dem Auftraggeber eine Entscheidung über die Abnahme der Software zu ermöglichen.

Es werden typische Nutzungsszenarien aus der Perspektive der Endnutzer simuliert. Die Auswertung basiert auf den in Abschnitt 3.3 Testfälle beschriebenen Einzelschritten, erwarteten Resultaten und Kriterien zur Bewertung der Testergebnisse.

Die Tests werden manuell auf einem Standard-Testsystem mit Windows 10 durchgeführt. Jeder Testfall wird separat dokumentiert und mit dem Ergebnis (bestanden/nicht bestanden) versehen. Im Abnahmeprotokoll werden Abweichungen, Fehler oder unerwartete Systemverhalten festgehalten.

Für die Systemabnahme durch den Auftraggeber ist es notwendig, dass alle Muss-Testfälle erfolgreich durchgeführt wurden.

3.1 Zu testende Anforderungen

Im Rahmen der Abnahmetests werden die folgenden funktionalen Anforderungen überprüft. Sie sind essenziell für die Funktionalität der Anwendung und wurden aus dem Pflichtenheft abgeleitet:

Nr	Anforderung	Testfälle	Kommentar
1	<Anlegen und Konfigurieren einer Szene (<F10>, <F11>, <F12>, <F13>, <F14>)>	<T100>	Basisfunktion zur Erstellung und Vorbereitung einer Show
2	<Starten und Beobachten der Simulation (<F20>)>	<T200>	Visualisierung und Ablaufprüfung der choreografierten Szenen
3	<Speichern und Laden von Projekten (<F30>)>	<T300>	Sicherstellung der Datenpersistenz
4	<Automatische Erkennung von Eingabefehlern in Szenen (<F40>)>	<T400>	Erhöht Bedienkomfort und reduziert Fehler beim Speichern oder Simulieren

3.2 Testverfahren

Die Abnahmetests für SkyOrchestrator erfolgen durch manuelle, dynamische Black-Box-Tests. Der Schwerpunkt liegt auf der Überprüfung der Funktionen, die für den Benutzer sichtbar sind, ohne Berücksichtigung des Quellcodes oder interner Programmstrukturen.

Die Testfälle basieren auf typischen Nutzungssituationen des Endanwenders. Die Ausführung findet auf einem Standard-Testsystem mit Windows 11 sta. In der finalen GUI der Anwendung wird getestet.

Bei jedem Testfall werden die folgenden Aspekte kontrolliert:

1. Richtigkeit der Ein- und Ausgaben
2. Antwort auf falsche Eingaben
3. Visuelles Nutzerverhalten im Interface
4. Logdatei-Einträge
5. Verhalten des Systems bei Speicherung und Simulation

Die Bewertung erfolgt nach dem „Bestanden / Nicht bestanden“-Prinzip, das sich auf klar definierte Kriterien im jeweiligen Testfall stützt.

3.2.1 Testskripte

Bei der Durchführung der Abnahmetests kommen keine automatisierten Testskripte zum Einsatz. Die Durchführung aller Testfälle erfolgt manuell, basierend auf den in Abschnitt 3.3 erläuterten Einzelschritten.

Die Testergebnisse werden aufgezeichnet, indem die Benutzeroberfläche visuell kontrolliert, das Verhalten des Systems beobachtet und Logdateien geprüft werden.

Sollte es während eines Tests zu einem Fehler kommen, wird dieser nachvollziehbar im Testprotokoll festgehalten. Eine strukturierte Erfassung erfolgt in tabellarischer Form zur Nachverfolgbarkeit.

3.3 Testfälle

Im Folgenden werden die Abnahmetestfälle für das System **SkyOrchestrator** beschrieben. Jeder Testfall bezieht sich auf eine im Pflichtenheft festgelegte Funktion und prüft, ob diese unter realistischen Bedingungen korrekt ausgeführt wird.

Die Testfälle sind nummeriert ($\langle T100 \rangle$ bis $\langle T400 \rangle$) und enthalten folgende Angaben:

- Ziel des Tests
- Betroffene Funktionen
- Pass/Fail-Kriterien
- Vorbedingungen
- Einzelschritte und erwartete Ausgabe
- Beobachtungen, Besonderheiten und Abhängigkeiten

3.3.1 Testfall $\langle T100 \rangle$ - Szene erstellen und speichern

Ziel

Überprüfung, ob ein neue Szene mit allen notwendigen Metadaten erstellt, konfiguriert und erfolgreich gespeichert werden kann.

Objekte/Methoden/Funktionen

$\langle F10 \rangle$ Show konfigurieren, $\langle F11 \rangle$ Szene anlegen, $\langle F12 \rangle$ Formation definieren, $\langle F13 \rangle$ Licht-/Zeiteffekte, $\langle F14 \rangle$ Szene speichern; RM1, RM2, RM4, RM6

Pass/Fail Kriterien

Bestanden: Die Szene wird erfolgreich gespeichert und erscheint korrekt in der Szenenübersicht

Nicht bestanden: Die Szene wird nicht gespeichert, oder es erscheint eine Fehlermeldung bei gültigen Eingaben.

Vorbedingung

Die Software ist gestartet.

Ein Projekt wurde neu erstellt oder geöffnet.

Einzelschritte:

1. In der Anwendung auf „Neue Szene erstellen“ klicken
2. Szenenname, Dauer und Anzahl der Drohnen eingeben
3. Formation und Bewegungsmuster auswählen
4. Auf „Szene speichern“ klicken

Ausgabe:

Die Szene erscheint in der Szenenliste mit allen eingegebenen Parametern.

Beobachtungen / Log / Umgebung

Die GUI zeigt die Szene korrekt an.

In der Logdatei steht z.B.: Szene erfolgreich gespeichert.

Besonderheiten

Gültige Eingaben erforderlich (z.B. keine leeren Felder).

Abhängigkeiten

Keine

3.3.2 Testfall $\langle T200 \rangle$ - Simulation einer Drohnenshow

Ziel

Überprüfung, ob die gespeicherte Szene korrekt in der 3D-Umgebung simuliert wird.

Objekte/Methoden/Funktionen

$\langle F20 \rangle$ Simulation starten RM3, RM6, RS1

Pass/Fail Kriterien

Bestanden: Die Simulation startet fehlerfrei, läuft flüssig ab und zeigt alle konfigurierten Bewegungen und Effekte.

Nicht bestanden: Die Simulation bleibt hängen, zeigt falsche Abläufe oder gibt Fehler aus.

Vorbedingung

Mindestens eine vollständige Szene ist gespeichert.

Einzelschritte

1. Szene aus der Übersicht auswählen.

2. „Simulation starten“ klicken.

Ausgabe: Die Szene wird in Echtzeit visualisiert, inklusive Drohnenbewegungen und Lichtanimationen.

Beobachtungen / Log / Umgebung

3D-Simulation läuft sichtbar.

Logeintrag: Simulation erfolgreich ausgeführt.

Besonderheiten

Simulation sollte auf einem System mit ausreichender Grafikleistung getestet werden.

Abhängigkeiten

Abhängig von Testfall $\langle T100 \rangle$

3.3.3 Testfall <T300> – Projekt speichern und laden

Ziel:

Überprüfung, ob ein Projekt korrekt gespeichert und anschließend fehlerfrei wieder geladen werden kann.

Objekte/Methoden/Funktionen:

<F30> Projekt speichern und laden; RS3,RM4,RM5

Pass/Fail Kriterien:

- **Bestanden:** Das Projekt wird vollständig gespeichert und kann nach dem Neustart der Anwendung ohne Datenverlust geladen werden.
- **Nicht bestanden:** Projektdatei wird nicht erstellt, beim Laden treten Fehler auf oder Daten (z.B. Szenen) fehlen.

Vorbedingung:

Ein Projekt mit mindestens einer vollständig konfigurierten Szene ist geöffnet.

Einzelschritte:

1. Auf „Projekt speichern“ klicken
2. Einen Dateinamen eingeben (z.B. `abendshow1`)
3. Anwendung schließen
4. Anwendung neu starten
5. Über „Projekt laden“ die zuvor gespeicherte Datei auswählen

Ausgabe:

Nach dem Laden erscheinen alle Szenen und Konfigurationen im gleichen Zustand wie vor dem Speichern.

Beobachtungen / Log / Umgebung:

Die GUI zeigt die gespeicherten Szenen korrekt an. In der Logdatei steht z.B.: `Projekt erfolgreich geladen`.

Besonderheiten:

Test kann sowohl mit manuellem Speichern als auch mit automatischer Speicherfunktion (falls vorhanden) wiederholt werden.

Abhängigkeiten:

Abhängig von Testfall <T100>

3.3.4 Testfall <T400> – Automatische Erkennung von Eingabefehlern

Ziel:

Überprüfung, ob das System ungültige oder unvollständige Eingaben beim Anlegen oder Speichern einer Szene korrekt erkennt und eine verständliche Fehlermeldung ausgibt.

Objekte/Methoden/Funktionen:

<F40> Eingabevalidierung RM1, RM4, RM7

Pass/Fail Kriterien:

- **Bestanden:** Das System erkennt die fehlerhafte Eingabe und verhindert das Speichern. Eine verständliche Fehlermeldung wird angezeigt.
- **Nicht bestanden:** Die Szene wird trotz falscher oder unvollständiger Angaben gespeichert, oder es erfolgt keine Rückmeldung an den Benutzer.

Vorbedingung:

Der Benutzer befindet sich im Formular zum Anlegen einer neuen Szene.

Einzelschritte:

1. Auf „Neue Szene erstellen“ klicken
2. Szenenname eingeben, aber Dauer oder Anzahl Drohnen leer lassen
3. Auf „Szene speichern“ klicken

Ausgabe:

Das System zeigt eine Fehlermeldung wie: "Bitte alle Pflichtfelder ausfüllen." Die Szene wird nicht gespeichert.

Beobachtungen / Log / Umgebung:

In der GUI erscheint eine Warnung. In der Logdatei steht z.B.: **Speichern abgebrochen - Eingabefehler.**

Besonderheiten:

Pflichtfelder (z.B. Szenenname, Dauer, Drohnenanzahl) müssen vollständig und plausibel befüllt sein. Grenzwerte (z.B. negative Werte, null Drohnen) sollen ebenfalls validiert werden.

Abhängigkeiten:

Keine

4 Integrationstest

Der Integrationstest überprüft das Zusammenspiel der Hauptkomponenten von SkyOrchestrator. Ziel ist es, die Kommunikation und Datenflüsse zwischen den funktionalen Modulen zu validieren, wie sie im Pflichtenheft und technischen Entwurf spezifiziert sind. Der Fokus liegt auf Schnittstellenverhalten, korrekter Datenweitergabe und der Reaktion des Systems auf Benutzeraktionen.

Die Tests werden manuell in einer Testumgebung mit Windows 11, Python 3.13 und PyBullet durchgeführt. Als Testmethodik kommt das Bottom-Up-Prinzip zur Anwendung. Getestet werden jeweils zwei oder mehr Komponenten im Zusammenspiel anhand realistischer Nutzungsszenarien.

4.1 Zu testende Komponenten

Nr	Komponenten	Testfälle	Kommentar
1	<C10> GUI + <C20> ShowKonfigurator	<T500>	
2	<C20> ShowKonfigurator + <C60> (Simulation)	<T510>	
3	<C10> GUI + <C60> (Simulation)	<T520>	
4	<C10> GUI + <C30> Projektmanagement	<T530>	
5	<C50> JSONImporter + <C60> Simulation	<T540>	Import von Showdaten aus JSON-Dateien (optional)
6	<C20> ShowKonfigurator + C70 Validierung	<T550>	
7	<C20> ShowKonfigurator + <C90> Szenenmanager + <C30>/<C60>	<T560>	

4.2 Testverfahren

Die Integrationstests werden manuell als dynamische Black-Box-Tests durchgeführt. Die Durchführung erfolgt nach dem Bottom-Up-Prinzip. Ziel ist die Überprüfung der Zusammenarbeit mehrerer Komponenten auf funktionaler Ebene. Die Tests orientieren sich an realitätsnahen Nutzungsszenarien und werden über die grafische Benutzeroberfläche und Konsolenausgaben beobachtet.

4.2.1 Testskripte

Es werden keine automatisierten Testskripte verwendet. Die Tests werden manuell anhand definierter Testfälle durchgeführt. Die Ergebnisse werden schriftlich dokumentiert. Bei Fehlern oder Abweichungen erfolgt eine ergänzende Dokumentation mit Screenshots, Logdateien oder Kommentaren.

4.3 Testfälle

4.3.1 Testfall $\langle T500 \rangle$ - Komponente $\langle C10+C20 \rangle$

Ziel: Überprüfung der Übergabe von Benutzereingaben aus der GUI an das Show-Konfigurationsmodul.

Objekte/Methoden/Funktionen:: `createScene()`, `saveScene()`

Pass/Fail-Kriterien: Szene erscheint korrekt in der Übersicht und wird gespeichert / Fehler beim Speichern oder fehlende Eingaben führen zu Misserfolg.

Vorbedingung: Projekt ist geöffnet.

Einzelschritte:

1. „Neue Szene“ in GUI starten
2. Drohnenanzahl und Formation eingeben
3. Lichteffekt definieren
4. Szene speichern
5. Szenenliste prüfen

Beobachtungen / Log / Umgebung :Neue Szene erscheint in der GUI unter „Szenenübersicht“

Besonderheiten: Pflichtfelder validieren (z.B. keine leere Drohnenanzahl)

Abhängigkeiten: Keine

4.3.2 Testfall $\langle T510 \rangle$ - Komponente $\langle C20+C60 \rangle$

Ziel: Szenenübergabe vom ShowKonfigurator an die Simulation

Objekte/Methoden/Funktionen:: `prepareForSimulation()`, `simulateScene()`

Pass/Fail-Kriterien: Übergabe erfolgt ohne Fehler / Simulationsmodul reagiert nicht oder bricht ab

Vorbedingung: Szene wurde in $\langle T500 \rangle$ erstellt

Einzelschritte:

1. Szene aus Liste auswählen
2. Simulation vorbereiten starten

Beobachtungen / Log / Umgebung : Übergabebestätigung im Log, keine Fehlermeldung

Besonderheiten: Log prüfen

Abhängigkeiten: $\langle T500 \rangle$

4.3.3 Testfall $\langle T520 \rangle$ - Komponente $\langle C10+C60 \rangle$

Ziel: Start der Simulation aus der Benutzeroberfläche

Objekte/Methoden/Funktionen: startSimulation()

Pass/Fail-Kriterien: Simulation läuft und zeigt Szene korrekt / Fehler oder fehlende Darstellung.

Vorbedingung: Szene wurde übergeben (T510).

Einzelschritte:

1. Szene wählen
2. Simulation starten
3. Animation beobachten

Beobachtungen / Log / Umgebung :Echtzeitanimation in 3D-Fenster (PyBullet)

Besonderheiten:3D-Fenster muss sichtbar sein

Abhängigkeiten: $\langle T510 \rangle$

4.3.4 Testfall $\langle T530 \rangle$ - Komponente $\langle C10+C30 \rangle$

Ziel: Projektzustand über GUI sichern und wiederherstellen

Objekte/Methoden/Funktionen:: `saveProject()`, `loadProject()`

Pass/Fail-Kriterien: Szenen und Daten werden korrekt geladen / Projekt lädt nicht oder ist unvollständig

Vorbedingung: Projekt mit mindestens einer Szene ist aktiv

Einzelschritte:

1. Projekt speichern
2. App schließen
3. App erneut starten
4. Projekt laden

Beobachtungen / Log / Umgebung : Szenen erscheinen wie vor dem Speichern

Besonderheiten: Projektname darf keine Sonderzeichen enthalten

Abhängigkeiten: $\langle T500 \rangle$

4.3.5 Testfall $\langle T540 \rangle$ - Komponente $\langle C50+C60 \rangle$

Ziel: Szenen aus externen JSON-Dateien importieren

Objekte/Methoden/Funktionen:: importJSON(), parseScene()()

Pass/Fail-Kriterien: Szene wird korrekt übernommen / Strukturfehler oder Abbruch

Vorbedingung: Gültige JSON-Datei liegt vor

Einzelschritte:

1. Datei wählen
2. Import starten
3. Szene prüfen

Beobachtungen / Log / Umgebung : Szene erscheint in der Szenenliste

Besonderheiten: Test nicht durchgeführt, Funktion nicht implementiert

Abhängigkeiten: Keine

4.3.6 Testfall $\langle T550 \rangle$ - Komponente $\langle C20+C70 \rangle$

Ziel: Eingaben prüfen, Fehler zurückmelden.

Objekte/Methoden/Funktionen:: `validateScene()`, `showErrorPopup()`

Pass/Fail-Kriterien: Fehler wird erkannt, Popup erscheint/ Ungültige Szene wird trotzdem akzeptiert.

Vorbedingung: Neue Szene mit leerem Pflichtfeld

Einzelschritte:

1. Szene mit leerer Drohnenanzahl anlegen
2. Speichern klicken

Beobachtungen / Log / Umgebung : GUI zeigt Fehlermeldung an

Besonderheiten: Validierung muss sofort reagieren

Abhängigkeiten: $\langle T500 \rangle$

4.3.7 Testfall $\langle T560 \rangle$ - Komponente $\langle C20+C30+C60+C90 \rangle$

Ziel: Prüfung, ob mehrere Szenen gespeichert, geladen und simuliert werden könne

Objekte/Methoden/Funktionen:: `saveSceneList()`, `loadSceneList()`, `selectScene()`

Pass/Fail-Kriterien: Beide Szenen verfügbar und korrekt simuliert / Szenen fehlen oder falsche Daten

Vorbedingung: Projekt mit mindestens zwei Szenen

Einzelschritte:

1. Zwei Szenen anlegen
2. Projekt speichern
3. App neustarten
4. Projekt laden
5. Beide Szenen simulieren

Beobachtungen / Log / Umgebung : Beide Szenen erscheinen, können separat gestartet werden

Besonderheiten: Reihenfolge der Szenen prüfen

Abhängigkeiten: $\langle T500 \rangle$, $\langle T530 \rangle$

5 Unit-Tests

Die Unit-Tests dienen der gezielten Überprüfung einzelner Softwarekomponenten von SkyOrchestrator auf korrekte Funktion. Ziel ist es, Fehler möglichst frühzeitig im Entwicklungsprozess zu erkennen und zu beheben, sodass spätere Integrations- und Systemtests auf einer stabilen Basis aufbauen können.

Im Fokus stehen dabei isolierte Module, Klassen und Funktionen, die unabhängig von anderen Systemteilen getestet werden. Die Tests werden manuell anhand definierter Testfälle durchgeführt, um eine systematische und nachvollziehbare Überprüfung zu gewährleisten.

5.1 Zu testende Komponenten

Im Rahmen der Unit-Tests werden die folgenden Kernkomponenten einzeln betrachtet. Die Tabelle zeigt die jeweiligen Komponenten, zugehörige Testfälle und einen kurzen Kommentar zum Testschwerpunkt.

Nr	Komponenten	Testfälle	Kommentar
1	⟨C20⟩ ShowKonfigurator	⟨T600⟩, ⟨T601⟩	Szenen anlegen, bearbeiten, löschen
2	⟨C30⟩ Projektmanagement	⟨T610⟩, ⟨T611⟩	Projekt speichern und laden
3	⟨C60⟩ Simulation	⟨T620⟩	Übergabe und Ablauf von Szenen in der Simulation
4	⟨C70⟩ Validierung	⟨T630⟩	Prüfung von Eingaben und Fehlererkennung
5	⟨C80⟩ GUI	<i>xx</i>	Prüfung von der Funktionalität der GUI

5.2 Testverfahren

Die Unit-Tests werden manuell anhand der zuvor definierten Testfälle durchgeführt. Für jede zu testende Funktionseinheit werden gezielt Testfälle entwickelt, die sowohl typische als auch fehlerhafte Eingaben abdecken. Die erwarteten Ausgaben werden mit den tatsächlich beobachteten Ergebnissen verglichen und schriftlich dokumentiert. Bei Abweichungen oder Fehlern erfolgt eine ergänzende Dokumentation, beispielsweise durch Screenshots, Logdateien oder erläuternde Kommentare. Die Tests werden regelmäßig wiederholt, um die Qualität der Software kontinuierlich sicherzustellen.

5.2.1 Testskripte

Automatisierte Testskripte kommen im Rahmen der Unit-Tests nicht zum Einsatz. Die Durchführung erfolgt ausschließlich manuell auf Basis der definierten Testfälle. Alle Ergebnisse werden schriftlich protokolliert. Bei auftretenden Fehlern oder Abweichungen werden zusätzliche Informationen wie Screenshots, Logauszüge oder erläuternde Kommentare zur besseren Nachvollziehbarkeit dokumentiert.

5.3 Testfälle

Im Folgenden sind beispielhafte Unit-Testfälle dokumentiert. Die Nummerierung beginnt ab T600, um Überschneidungen mit anderen Testarten zu vermeiden.

5.3.1 Testfall $\langle T600 \rangle$ - ShowKonfigurator: Szene anlegen

Ziel

Überprüfung, ob eine neue Szene korrekt erstellt und in der Szenenliste gespeichert wird.

Objekte/Methoden/Funktionen

`add_scene()`

Pass/Fail Kriterien

Die Szene ist nach dem Hinzufügen in der Szenenliste auffindbar und enthält die übergebenen Metadaten.

Vorbedingung

Die Szenenliste ist initial leer.

Einzelschritte

1. Neue Szene mit Name und Metadaten anlegen.
2. Szenenliste abfragen.

Beobachtungen / Log / Umgebung

Prüfen, ob Szene korrekt in der Liste enthalten ist.

Besonderheiten

Keine

Abhängigkeiten

Keine

5.3.2 Testfall $\langle T601 \rangle$ - ShowKonfigurator: Szene bearbeiten

Ziel

Überprüfung, ob eine bestehende Szene korrekt bearbeitet und die Änderungen gespeichert werden.

Objekte/Methoden/Funktionen

`edit_scene()`

Pass/Fail Kriterien

Die Änderungen an der Szene sind nach dem Speichern in der Szenenliste sichtbar.

Vorbedingung

Mindestens eine Szene existiert.

Einzelschritte

1. Vorhandene Szene auswählen.
2. Änderungen vornehmen (z.B. Name, Metadaten).
3. Speichern.
4. Szenenliste abfragen.

Beobachtungen / Log / Umgebung

Geänderte Daten sind sichtbar, keine Fehlermeldung.

Besonderheiten

Prüfen, ob auch ungültige Änderungen abgefangen werden.

Abhängigkeiten

Testfall T600 (Szene anlegen)

5.3.3 Testfall $\langle T610 \rangle$ - Projektmanagement: Projekt speichern und laden

Ziel

Überprüfung, ob ein Projekt korrekt gespeichert und anschließend fehlerfrei wieder geladen werden kann.

Objekte/Methoden/Funktionen

`save_project()`, `load_project()`

Pass/Fail Kriterien

Das Projekt wird vollständig gespeichert und kann ohne Datenverlust geladen werden.

Vorbedingung

Ein Projekt mit mindestens einer Szene ist geöffnet.

Einzelschritte

1. Projekt speichern.
2. Anwendung neu starten.
3. Projekt laden.

Beobachtungen / Log / Umgebung

Nach dem Laden erscheinen alle Szenen und Konfigurationen wie vor dem Speichern.

Besonderheiten

Test kann mit verschiedenen Projektgrößen wiederholt werden.

Abhängigkeiten

Keine

5.3.4 Testfall $\langle T611 \rangle$ - Projektmanagement: Projekt löschen

Ziel

Überprüfung, ob ein bestehendes Projekt gelöscht werden kann und nicht mehr in der Projektliste erscheint.

Objekte/Methoden/Funktionen

`delete_project()`

Pass/Fail Kriterien

Das Projekt ist nach dem Löschen nicht mehr verfügbar und kann nicht geladen werden.

Vorbedingung

Mindestens ein Projekt ist gespeichert.

Einzelschritte

1. Zu löschendes Projekt auswählen.
2. Löschfunktion ausführen.
3. Projektliste aktualisieren.
4. Versuchen, das gelöschte Projekt zu laden.

Beobachtungen / Log / Umgebung

Projekt ist aus der Liste entfernt, Ladevorgang schlägt fehl.

Besonderheiten

Prüfen, ob versehentliches Löschen abgefangen wird (z.B. durch Rückfrage).

Abhängigkeiten

Testfall T610 (Projekt speichern und laden)

5.3.5 Testfall $\langle T620 \rangle$ - Simulation: Szenenübergabe

Ziel

Überprüfung, ob eine konfigurierte Szene korrekt an das Simulationsmodul übergeben und dort gestartet wird.

Objekte/Methoden/Funktionen

`start_simulation()`

Pass/Fail Kriterien

Die Simulation startet mit den übergebenen Szenendaten und läuft ohne Fehler an.

Vorbedingung

Mindestens eine Szene ist vollständig konfiguriert.

Einzelschritte

1. Szene auswählen.
2. Simulation starten.
3. Simulationsverlauf beobachten.

Beobachtungen / Log / Umgebung

Simulation läuft wie erwartet, Logeintrag bestätigt Start.

Besonderheiten

Test auf verschiedenen Szenenkonfigurationen wiederholen.

Abhängigkeiten

Testfall T600 (Szene anlegen)

5.3.6 Testfall $\langle T630 \rangle$ - Validierung: Eingabefehler erkennen

Ziel

Überprüfung, ob ungültige oder unvollständige Eingaben korrekt erkannt und eine Fehlermeldung ausgegeben wird.

Objekte/Methoden/Funktionen

`validate_scene_input()`

Pass/Fail Kriterien

Fehlerhafte Eingaben werden erkannt und eine verständliche Fehlermeldung wird zurückgegeben.

Vorbedingung

Das Validierungsmodul ist initialisiert.

Einzelschritte

1. Szene mit leerem Pflichtfeld anlegen.
2. Validierung ausführen.

Beobachtungen / Log / Umgebung

Rückgabe einer Fehlermeldung, keine Speicherung der Szene.

Besonderheiten

Verschiedene Fehlerarten testen (leere Felder, ungültige Werte).

Abhängigkeiten

Keine

——aufgabenstellung muss noch gelöscht werden—— In diesem Kapitel soll die Vorgehensweise im Unit-Test beschrieben werden. Die Bearbeitung soll genauso erfolgen, wie bereits in der Abnahmetestspezifikation der Abnahmetest. Achtet bitte darauf, dass die Nummerierung der Testfälle sich nicht mit der aus der Abnahmetestspezifikation überschneidet.

Testziel...

5.4 Zu testende Komponenten

Nr	Komponenten	Testfälle	Kommentar
1	<Ref. auf Komponente> Name der Komponente	<Referenz auf Testfall>	Kommentar
2	<Ref. auf Komponente> Name der Komponente	<Referenz auf Testfall>	

5.5 Testverfahren

...

5.5.1 Testskripte

...

5.6 Testfälle

5.6.1 Testfall $\langle T300 \rangle$ - Klasse x

Ziel

Objekte/Methoden/Funktionen

Pass/Fail Kriterien

Vorbedingung

Einzelschritte

Beobachtungen / Log / Umgebung

Besonderheiten

Abhängigkeiten

6 Glossar

Abnahmetest	Letzte Teststufe im V-Modell zur formellen Überprüfung der Funktionalität der Software anhand der im Pflichtenheft festgelegten Muss-Kriterien.
Black-Box-Test	Testverfahren, bei dem nur die Eingaben und Ausgaben betrachtet werden – ohne Wissen über die interne Implementierung.
Drohnenshow	Visuelle Show mit mehreren Drohnen, die sich koordiniert in Formationen bewegen und Lichteffekte anzeigen.
GUI	Grafische Benutzeroberfläche, über die Nutzer:innen mit der Software interagieren können.
Integrationstest	Teststufe zur Überprüfung des Zusammenspiels zwischen mehreren Softwarekomponenten.
Komponententest	Test einzelner Softwarebausteine (Units) auf korrekte Funktion.
Muss-Kriterium	Unverzichtbare Anforderungen, die für eine erfolgreiche Abnahme erfüllt sein müssen.
Pflichtenheft	Vertragliches Dokument, das die Anforderungen und Funktionen der zu entwickelnden Software festlegt.
Projektmanager	Modul zum Speichern, Laden und Verwalten von Projektdaten (Szenen, Metadaten etc.).
PyBullet	Physik-Engine für die Simulation von Bewegungen in einer 3D-Umgebung (hier für Drohnensimulation).
Qualitätsanforderung	Nicht-funktionale Anforderungen wie Effizienz, Fehlertoleranz oder Plattformkonformität.

ShowKonfigurator	Komponente zur Planung und Konfiguration von Drohnenszenen (z.B. Formationen, Lichteffekte).
Simulation	Darstellung und Ablaufenlassen einer geplanten Drohnenshow in einer virtuellen 3D-Umgebung.
Szenenmanager	Verwaltungseinheit zur Organisation und Speicherung einzelner Szenen innerhalb eines Projekts.
Systemtest	Überprüfung der Software als Ganzes auf Basis realer Nutzungsszenarien und Anforderungen.
Testfall	Detaillierte Beschreibung eines zu testenden Ablaufs mit Ziel, Eingaben, erwarteten Ergebnissen etc. (z.B. T100, T200,...).
Testskript	Schritt-für-Schritt-Anleitung zur Durchführung eines Tests (im vorliegenden Fall manuell, nicht automatisiert).
Testumgebung	Hardware- und Softwarekonfiguration, unter der Tests durchgeführt werden (hier z.B. Windows 11 + PyBullet).
Timeline-Komponente	Zeitsteuerungsmodul der GUI zur Festlegung von Start, Ende und Effekten einer Szene.
Validierung	Überprüfung auf Vollständigkeit und Plausibilität der Nutzereingaben in einer Szene.
V-Modell	Vorgehensmodell für die Softwareentwicklung mit sequentiellen Entwicklungs- und Teststufen.